

安全加固报告 FIX-08：命令执行与Shell调用

修复漏洞类型：命令执行与Shell调用

修复小组：天亮了么

修复时间：2026-08-27

对应漏洞报告：VULN-08-命令执行与Shell调用

修复负责人：蒲云楷

1. 基本信息

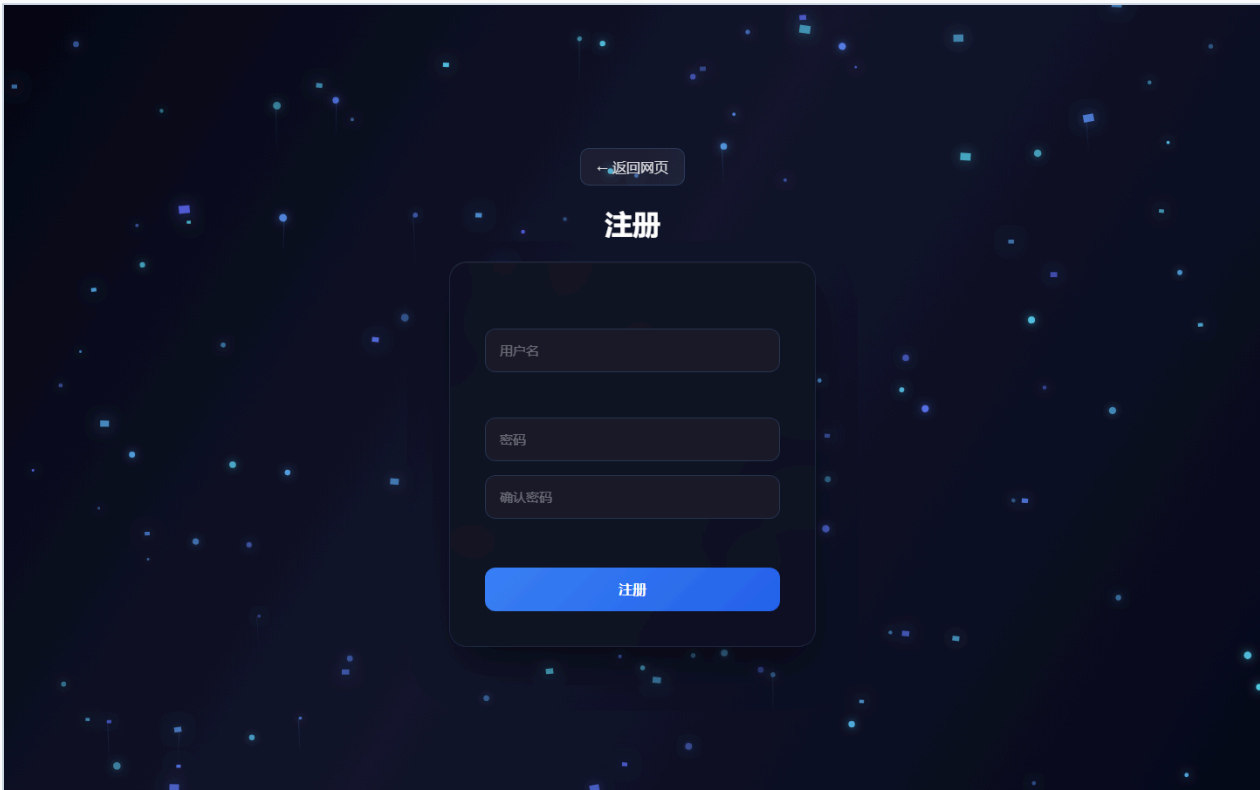
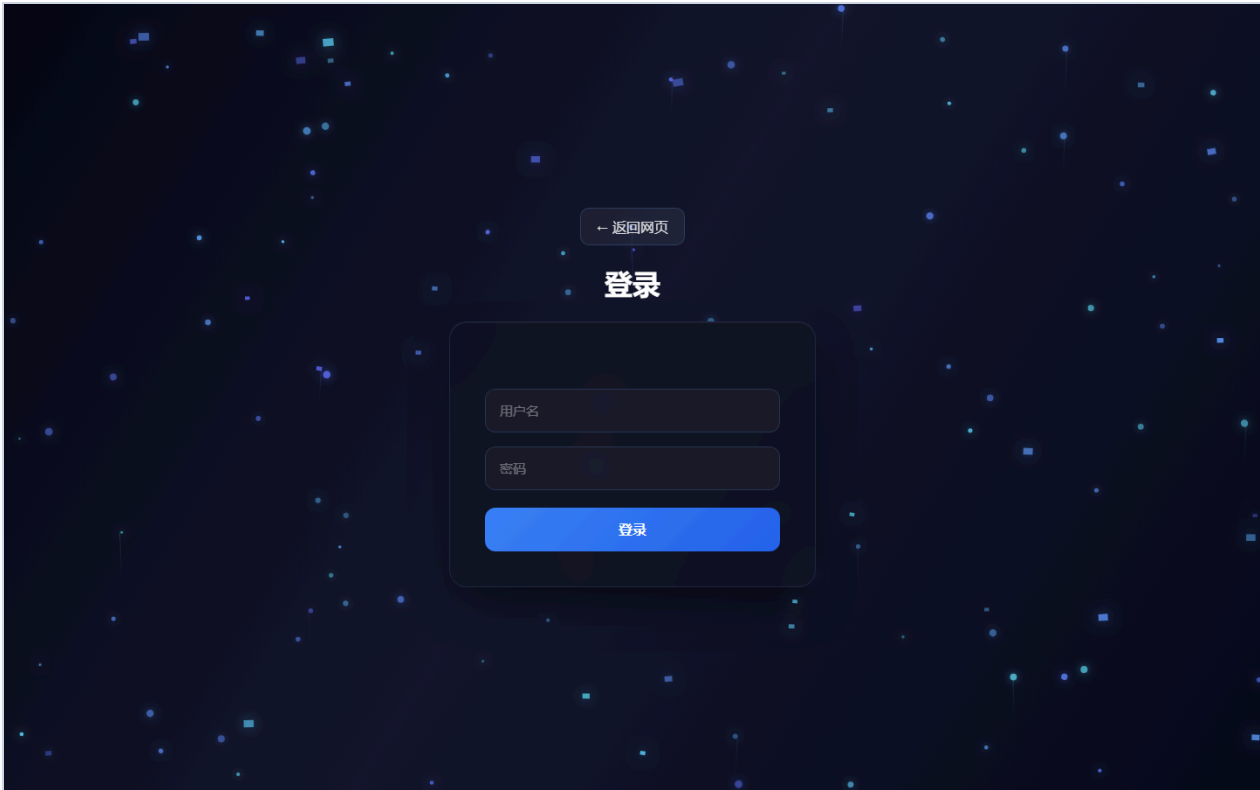
本报告按附录A记录阶段二任务8的修复和验证。

2. 漏洞复现情况

是否能按攻击方步骤复现：修复前基线已记录；修复后原始参数应被拦截或不产生越权效果。

复现环境：本机 127.0.0.1:5001。





上传文件

选择文件

Choose File

No file chosen

可选新文件名

留空则使用原文件名

上传文件

返回个人主页

修复代码截图（实际源文件与行号）

任务 08 修复代码证据

gets.py:1-25

```
1 import logging
2 import os
3 import secrets
4 import time
5 from functools import wraps
6
7 import pymysql
8 from flask import (
9     Flask,
10     abort,
11     jsonify,
12     redirect,
13     render_template,
14     request,
15     send_from_directory,
16     session,
17 )
18 from werkzeug.security import check_password_hash, generate_password_hash
19 from werkzeug.utils import secure_filename
20
21 app = Flask(__name__)
22 app.secret_key = os.environ.get("FLASK_SECRET_KEY") or secrets.token_hex(32)
23 app.config.update(
24     SESSION_COOKIE_HTTPONLY=True,
25     SESSION_COOKIE_SAMESITE="Lax",
```

gets.py:340-410

```
340     user_dir = os.path.join(app.config["UPLOAD_FOLDER"], session["username"])
341     os.makedirs(user_dir, exist_ok=True)
342     file.save(os.path.join(user_dir, filename))
343     security_logger.info("upload_success username=%s ip=%s filename=%s", session.get("username"), client_ip(), filename)
344     return jsonify({"success": True, "file": filename}), 200
345
346
347 @app.post("/api/files")
348 @api_login_required
349 @csrf_protected
350 def api_upload_file():
351     return _save_upload(request.files.get("file"), request.form.get("customName", ""))
352
353
354 @app.get("/listfiles")
355 def list_files():
356     if "username" not in session:
357         return redirect("/login")
358     return render_template("filelist.html", username=session["username"], files=[item["name"] for item in current_user_files()])
359
360
361 @app.get("/api/files")
362 @api_login_required
363 def api_list_files():
364     return jsonify({"files": current_user_files()})
365
366
367 def safe_user_file(filename):
368     filename = secure_filename(filename or "")
369     if not filename:
370         abort(400, description="文件名无效")
371     user_dir = os.path.join(app.config["UPLOAD_FOLDER"], session["username"])
372     path = os.path.join(user_dir, filename)
373     if not os.path.isfile(path):
374         abort(404, description="文件不存在")
375     return user_dir, filename
376
377
378 @app.get("/download")
379 def download_file():
380     if "username" not in session:
381         return "未登录", 401
382     user_dir, filename = safe_user_file(request.args.get("filename"))
383     return send_from_directory(user_dir, filename, as_attachment=True)
384
385
386 @app.get("/api/files/<path:filename>/download")
387 @api_login_required
388 def api_download_file(filename):
389     user_dir, filename = safe_user_file(filename)
390     return send_from_directory(user_dir, filename, as_attachment=True)
391
392
393 @app.route("/delete", methods=["GET", "DELETE"])
394 def delete_file():
395     if "username" not in session:
396         return "未登录", 401
397     supplied = request.args.get("csrf_token") or request.headers.get("X-CSRF-Token")
398     if not supplied or not secrets.compare_digest(supplied, session.get("csrf_token", "")):
399         return jsonify({"error": "请求验证失败, 请刷新页面后重试"}), 403
400     user_dir, filename = safe_user_file(request.args.get("filename"))
401     os.remove(os.path.join(user_dir, filename))
402     return redirect("/listfiles") if request.method == "GET" else jsonify({"success": True})
403
404
405 @app.delete("/api/files/<path:filename>")
406 @api_login_required
407 @csrf_protected
408 def api_delete_file(filename):
409     user_dir, filename = safe_user_file(filename)
410     os.remove(os.path.join(user_dir, filename))
```

3. 根因定位

相关文件：gets.py、相关模板和前端 API 客户端。

相关函数或接口：gets.py 全部系统调用点。

问题代码/根因：当前项目没有用户输入进入 Shell 的功能。

4. 修复措施

修改点1

修改文件： gets.py。

修改前逻辑： 缺少统一安全边界或校验。

修改后逻辑： 不添加危险命令接口；未来优先标准库，外部程序使用参数数组、shell=False、白名单和最小权限。

修改点2

修改文件： 相关 HTML/CSS/前端 API（按任务适用）。

修改后逻辑： 页面和客户端与服务端安全校验保持一致，不能替代服务端校验。

5. 修复后验证

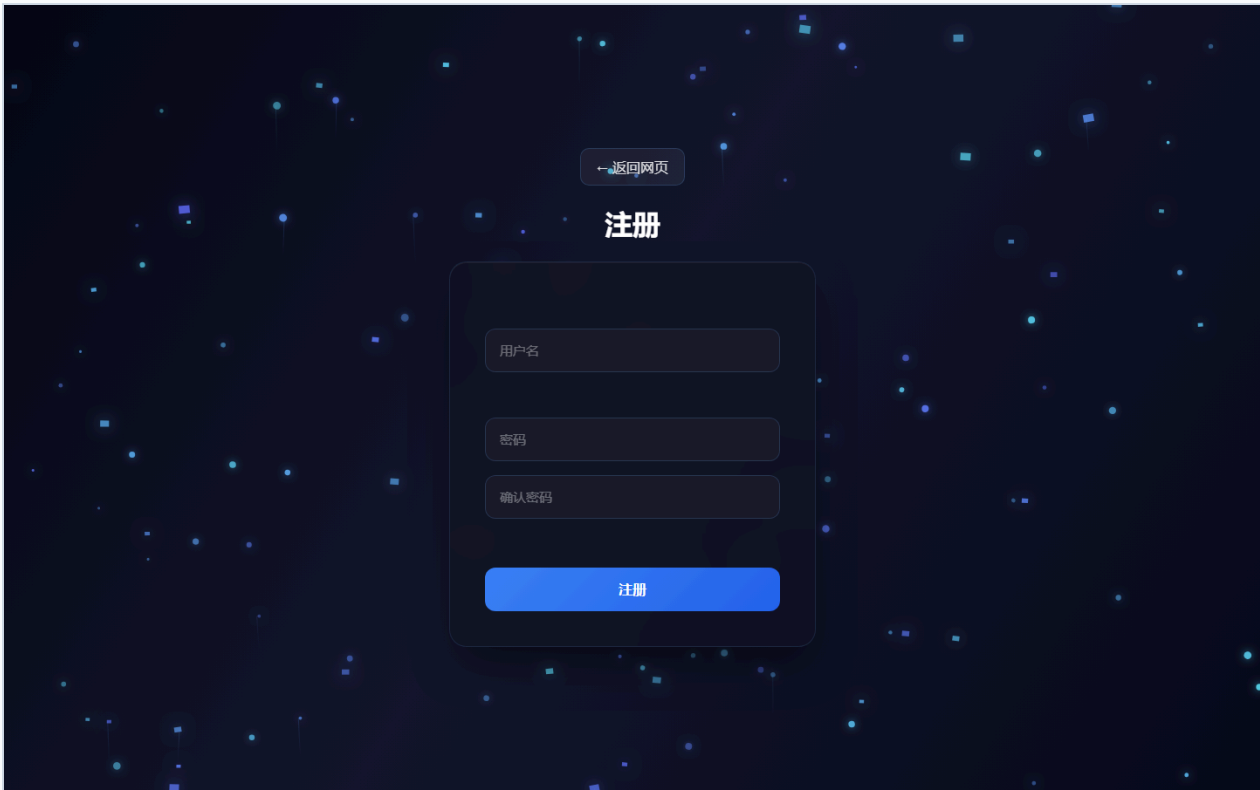
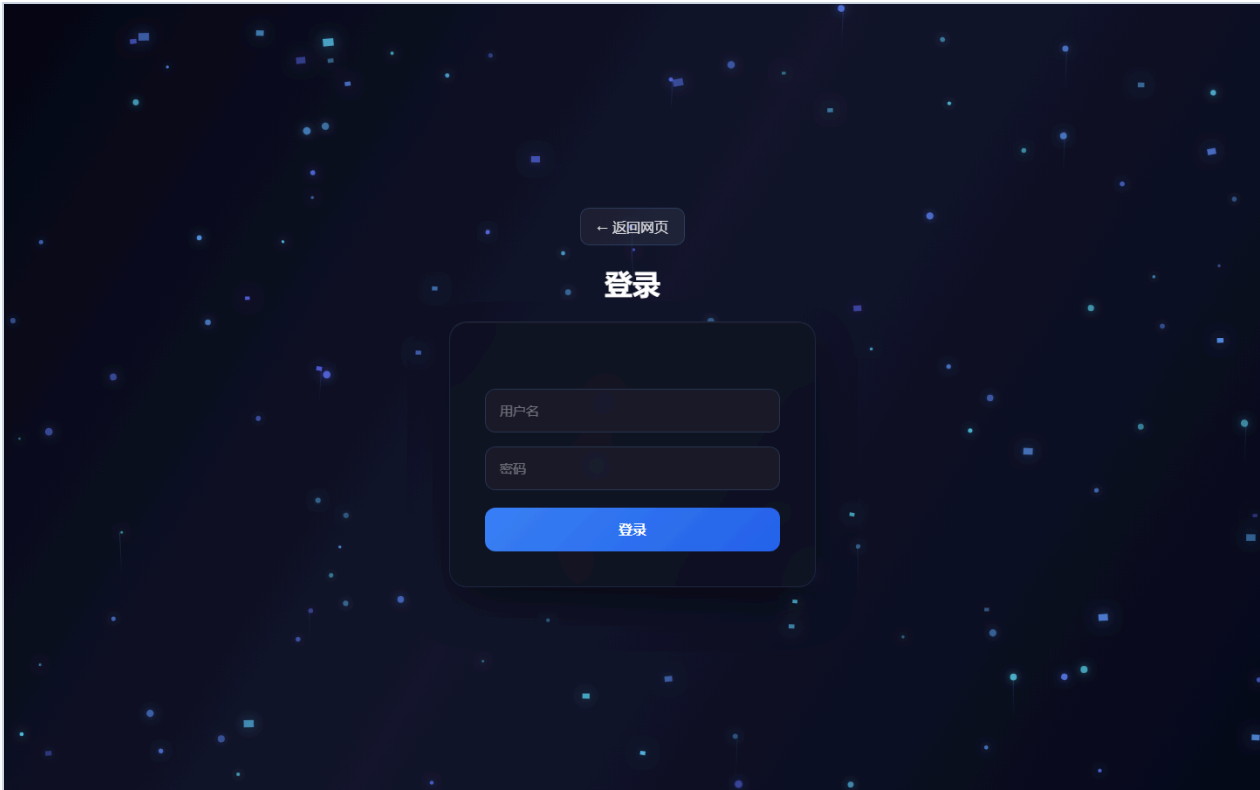
使用攻击方原始步骤重新测试：

1. 发送原始无害测试参数。
2. 观察状态码、页面、数据、文件和日志。
3. 确认原攻击失效并保存截图。

验证结果：审计未发现 os.system、os.popen、subprocess 或 shell=True；项目无可供探测的命令调用点。

原攻击是否失效：是（本机回归测试）





上传文件

选择文件

Choose File

No file chosen

可选新文件名

留空则使用原文件名

上传文件

返回个人主页

任务 08 修复代码证据

gets.py:1-25

```
1 import logging
2 import os
3 import secrets
4 import time
5 from functools import wraps
6
7 import pymysql
8 from flask import (
9     Flask,
10     abort,
11     jsonify,
12     redirect,
13     render_template,
14     request,
15     send_from_directory,
16     session,
17 )
18 from werkzeug.security import check_password_hash, generate_password_hash
19 from werkzeug.utils import secure_filename
20
21 app = Flask(__name__)
22 app.secret_key = os.environ.get("FLASK_SECRET_KEY") or secrets.token_hex(32)
23 app.config.update(
24     SESSION_COOKIE_HTTPONLY=True,
25     SESSION_COOKIE_SAMESITE="Lax",
```

gets.py:340-410

```
340     user_dir = os.path.join(app.config["UPLOAD_FOLDER"], session["username"])
341     os.makedirs(user_dir, exist_ok=True)
342     file.save(os.path.join(user_dir, filename))
343     security_logger.info("upload_success username=%s ip=%s filename=%s", session.get("username"), client_ip(), filename)
344     return jsonify({"success": True, "file": filename}), 200
345
346
347 @app.post("/api/files")
348 @api_login_required
349 @csrf_protected
350 def api_upload_file():
351     return _save_upload(request.files.get("file"), request.form.get("customName", ""))
352
353
354 @app.get("/listfiles")
355 def list_files():
356     if "username" not in session:
357         return redirect("/login")
358     return render_template("filelist.html", username=session["username"], files=[item["name"] for item in current_user_files()])
359
360
361 @app.get("/api/files")
362 @api_login_required
363 def api_list_files():
364     return jsonify({"files": current_user_files()})
365
366
367 def safe_user_file(filename):
368     filename = secure_filename(filename or "")
369     if not filename:
370         abort(400, description="文件名无效")
371     user_dir = os.path.join(app.config["UPLOAD_FOLDER"], session["username"])
372     path = os.path.join(user_dir, filename)
373     if not os.path.isfile(path):
374         abort(404, description="文件不存在")
375     return user_dir, filename
376
377
378 @app.get("/download")
379 def download_file():
380     if "username" not in session:
381         return "未登录", 401
382     user_dir, filename = safe_user_file(request.args.get("filename"))
383     return send_from_directory(user_dir, filename, as_attachment=True)
384
385
386 @app.get("/api/files/<path:filename>/download")
387 @api_login_required
388 def api_download_file(filename):
389     user_dir, filename = safe_user_file(filename)
390     return send_from_directory(user_dir, filename, as_attachment=True)
391
392
393 @app.route("/delete", methods=["GET", "DELETE"])
394 def delete_file():
395     if "username" not in session:
396         return "未登录", 401
397     supplied = request.args.get("csrf_token") or request.headers.get("X-CSRF-Token")
398     if not supplied or not secrets.compare_digest(supplied, session.get("csrf_token", "")):
399         return jsonify({"error": "请求验证失败, 请刷新页面后重试"}), 403
400     user_dir, filename = safe_user_file(request.args.get("filename"))
401     os.remove(os.path.join(user_dir, filename))
402     return redirect("/listfiles") if request.method == "GET" else jsonify({"success": True})
403
404
405 @app.delete("/api/files/<path:filename>")
406 @api_login_required
407 @csrf_protected
408 def api_delete_file(filename):
409     user_dir, filename = safe_user_file(filename)
410     os.remove(os.path.join(user_dir, filename))
```

6. 额外加固

统一错误提示、输入长度限制、安全日志、Cookie 安全属性、调试模式关闭和生产密钥配置要求。

7. 仍需改进

课程组别、负责人、Git tag 等元数据需在提交前填写；HTTPS 部署时启用 Secure Cookie；Web 服务器需确认上传目录不可执行。